ATC'26 · HONG KONG · NOVEMBER 2026

A Novel AI-Native GPU Kernel Programming Language for Autonomous Kernel Tuning

Heng Shi · Xiaofeng Guan · Sijie Wu · Enming Fan · Chenpeng Wu
Qiqi Gu · Jiahang Xie · Guoqing Bao · Jianguo Yao

Shanghai Enflame Intelligence & Shanghai Jiao Tong University

NOTES

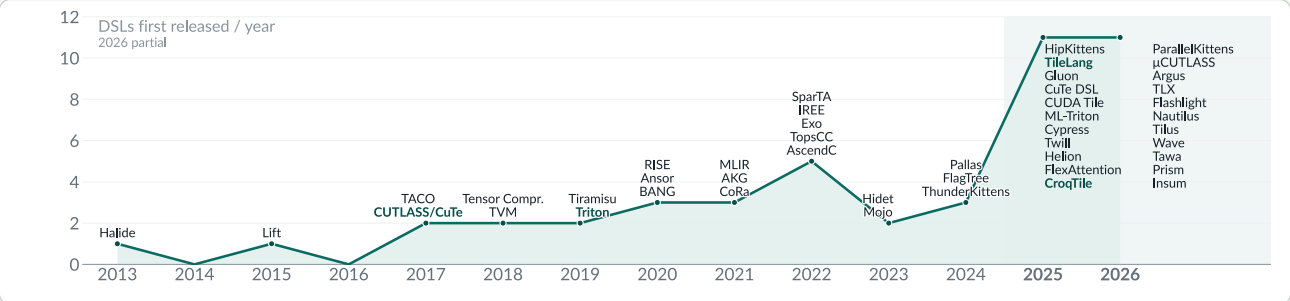
P1 · Title · 0:00–0:15

Hi everyone, I am Heng Shi from Enflame Technology and Shanghai Jiao Tong University. I am happy here to present our work on CroqTile, a new GPU programming language designed for autonomous kernel tuning. Let us get start with the motivation.

02 / 20

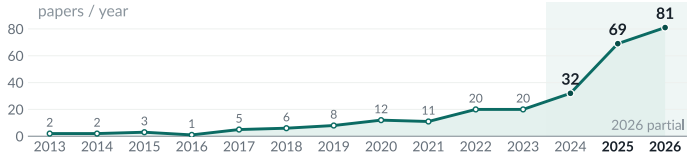
Operator work keeps growing

Model families converge. ***Kernel variants multiply.***



22/46 DSLS IN 2025–26

- **More programming substrates**
  46 DSLs in the survey corpus.
- **More operator work**
  276 optimization papers across 2013–26.



NOTES

P2 · Operator work keeps growing · 0:15–1:10

As you can see in the figures, the bottom one shows the number of works related to operator optimization and kernel tuning, and the the larger figure shows the number of programming models, domain-specific languages, and tools emerged for writing computing kernels.

From the data, we can easily to tell two things, on the one hand, the topic about kernel tuning has draw much attention than ever before, especially for the most recent 2-3 years. On the other hand, recently emerged DSLs seems all have much higher abstraction level for programming.

To me, this two phenomenon reveals one thing. the demand for operator development has largely increased.

Why is that? we know the model architecture are actually converged into several families, like encoder, decoder and diffusion model, so why the operator optimization efforts are somehow increased

03 / 20

Why kernels multiply

1	**Operator variants**	MHA · MQA · GQA · MLA · sliding-window · block-sparse Attention keeps branching
2	**Hardware diversity**	NVIDIA · AMD MI300X · Ascend · TPU · Trainium One kernel, many targets
3	**New GPU instructions**	Hopper: WGMMA + TMA → Blackwell: tcgen05 + TMEM New instructions, new dataflows
4	**Operator fusion**	Conv + BN + ReLU → fusion changes the kernel boundary Eliminate intermediate traffic

The maintenance burden multiplies. Operator variants × hardware targets × optimization choices.

NOTES

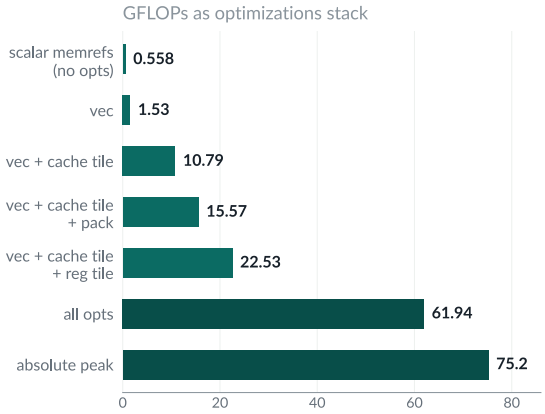
P3 · Why kernels multiply · 1:10–2:10
I think there are four reasons stack together.
Firstly, the operators themselves keep changes. we now have much more variants of one operator than the early year of deep learning. Take attn layers as example, you now have so many variants listed here.
Secondly, we have more hardware platforms than ever before, due to the demand for computing power. just like GPGPU, TPU, PIM, DSA and so many more.
And also, even within the same device itself, it still evolves and changes. This change will lead to things like new HW features, different ISAs, new programming abstractions, and of course different sweet zone for performance.
And lastly, you may notice that the amount of fused operators today are hugely increased with the evolution of AI inference infrastructure
all these together makes the effort for writing computing kernel become so expensive.

04 / 20

Kernel optimizations interact

COUPLED PERFORMANCE DECISIONS

- **Locality**
  Tiling · data reuse · L2 bypass / cache policy
- **Data movement**
  Swizzling · vectorization · async copies
- **Execution overlap**
  Multibuffering · warp specialization · scheduling
- **Compute & cooperation**
  Unrolling · approximation · sparsity · fusion
  MMA layouts · low-precision MMA · clusters / DSMEM



A larger tile improves reuse... ...but consumes more shared memory, limiting stages and overlap.

NOTES

P4 · Kernel optimizations interact · 2:10–3:10

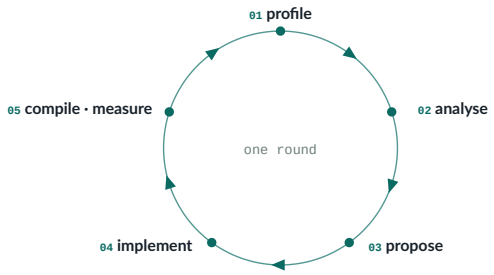
So of course, we would hope kernel tuning itself is not that hard. unfortunately, this is unrealistic. Let's look at how a single kenrel tuned, there will be so many things to consider. locality, bank conflict, instruction spill, memory hierarchy. For these concerns, we then designed many optimization techniques and tricks, like data Tiling, data reuse, swizzling, multibuffering, warp specialization.

So what does a optimised kernel looks like? just like this figure, you would have to find out a recipe of techniques and trick and combine them together, this recipe is never easy to find. because all these factors are coupled together, for example on GPU, a larger tile may improve locality, but can consume more shared memory per block, this will unfortunately limit the parallelism

So you can see, tradeoff is everywhere, so, how do we normally do to find this combination recipe?

05 / 20

Bottleneck-driven tuning



1. Profile
2. Identify bottleneck
3. Choose structural edit
4. Implement
5. Compile · measure · keep / reject

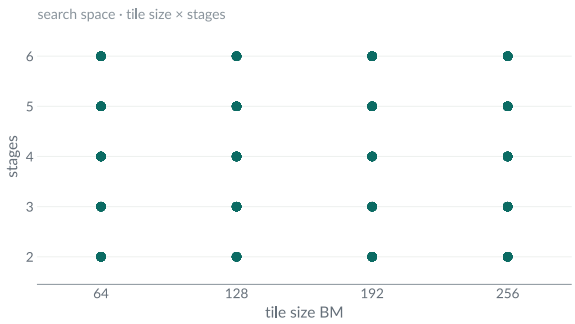
- **Example: tensor cores waiting for data**
  Introduce a producer–consumer pipeline.
- **Expert effort in every round**
  Interpret counters, redesign the kernel, validate the edit.

NOTES

P5 · Bottleneck-driven tuning · 3:10–3:50
The first way is probably familiar to you.
We can simply start from a naive version of implementation, profile it, find out its bottleneck, and pick up a trick for this bottleneck. by make this move iteratively like a loop. you will somehow approach a near-optimal version of kernel.
We can call this bottleneck-first approach or what ever you like. and this i s how human developer normally does. no matter you are an apprentice or experts. Experts may be able to skip iterations, make multiple changes on the kernel and see further, but anyway, they still follow this workflow.

06 / 20

Template search



`Tile size · stages · warps · swizzle`

- **Fixed skeleton**
  Choose the implementation structure up front.
- **Parameter search**
  Enumerate tile sizes, stages, warps and swizzles.
- **Filter, then benchmark**
  Reject resource violations; measure valid candidates.

Boundary: a new producer–consumer structure must first be exposed in the template.

NOTES

P6 · Template search · 3:50–4:55

The second way I would like to call it search-based approach. for this way, you will find familiar names, such as autotuning and loop synthesiser. Normally, We would write a template called kernel scheme with tunable parameters, like tile size, stages, and warp count.

As the figure shows, this is a search problem in discrete space, where each point is one concrete kernel implementaion. Having this search space, you can then use algorithms you like to approaching the optimal kernel. you can do genetic search, you can do monte carlo, you can do swarm search. This method was popular years ago, because they are the automation attempt for tuning kernels to save human efforts before AI agents pops in. and for this purpose, we also created tools for exposing more performance related factors to the syntax, and let you control. The famous name for these tools are halide IR, TVM and we call them scheduling DSL.

07 / 20

Compiler transformations

data tiling / blocking	→ **Loop tiling**
data stationary	→ **Cache staging**
multibuffering & prefetch	→ **Software pipelining**
loop unroll	→ **Unrolling**
vectorisation	→ **Vectorization**
loop fusion	→ **Fusion**
instruction scheduling	→ **Instruction scheduling**

MLIR · TVM · IREE

- **Schedule DSL**
  Express transformations in Halide / TVM TE.
- **Compiler passes**
  Encode legal rewrites over loops and dataflow.

New hardware, new structure: someone still needs to implement the corresponding transformations.

NOTES

P7 · Compiler transformations · 4:55–5:30

The third way is to put these optimization tricks into the compiler. As you can see here, data tiling becomes loop tiling, data reuse becomes cache staging, and multibuffering becomes software pipelining.

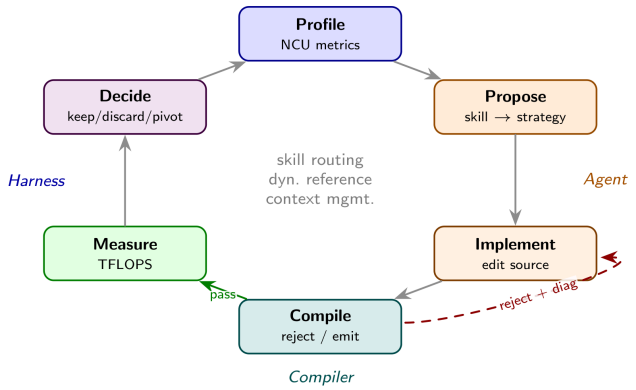
So we can describe a schedule, or let compiler passes apply these transformations. This is where tools like TVM and MLIR come in. But there is still human effort behind it. When new hardware needs a new kernel structure, somebody has to teach the compiler how to make that transformation.

08 / 20

Agents rewrite the kernel

PROFILE → EDIT → EVALUATE

Full source edits · structural + parametric



WORKFLOW	Planning · search · refinement
EVALS	Correctness · speed · generalization
TOOLING	Syntax–context · compiler–harness co-design
KNOWLEDGE	Skills · references · history

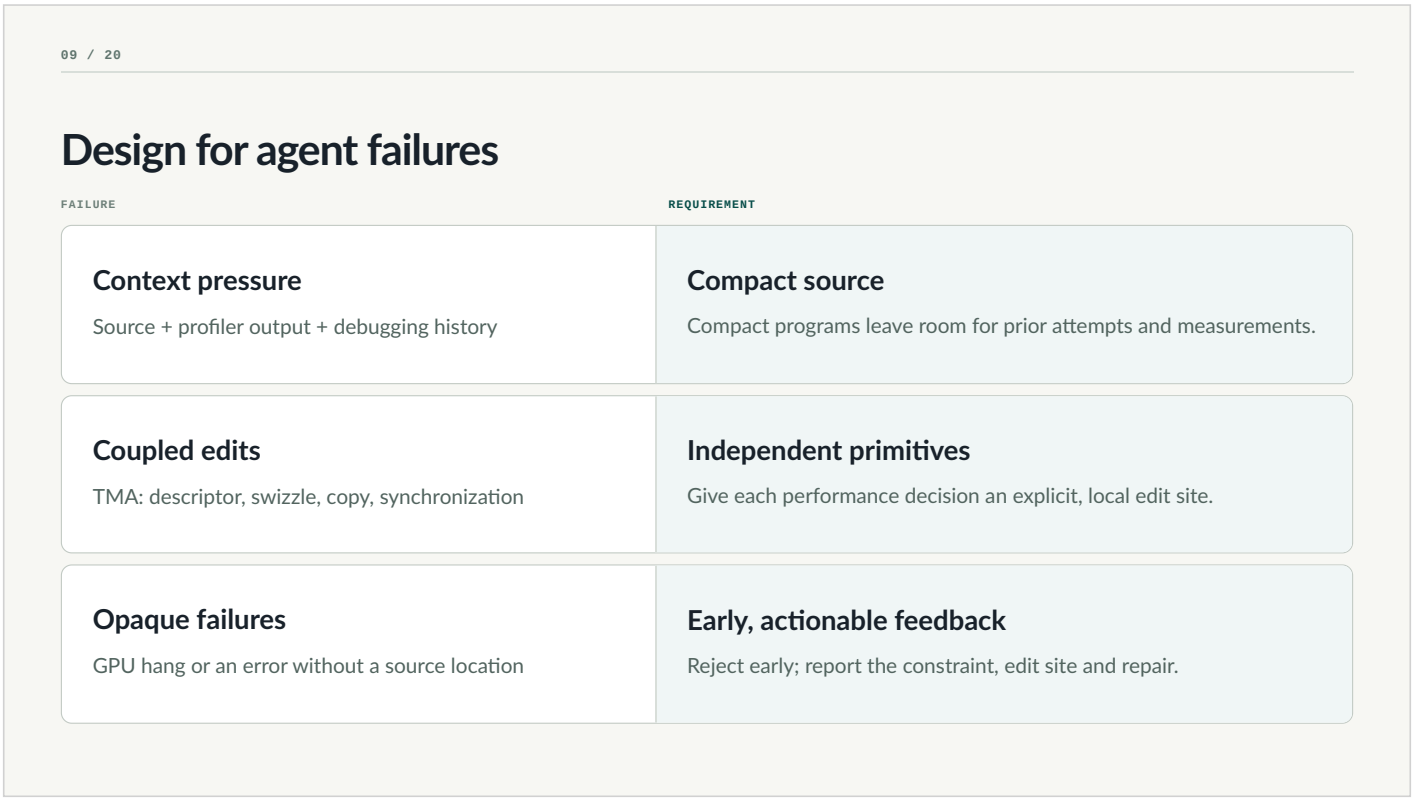
NOTES

P8 · Agents rewrite the kernel · 5:30–6:15

So now, with coding agents, we have another way to do this. Remember the bottleneck-first approach? We can put an agent into that loop, let it read the kernel, look at the profiler feedback, and pick up a trick to try.

The agent can change both the parameters and the kernel structure. And of course, there are many efforts on planning, evaluation, tools and knowledge, as listed here.

In our work, we want to design an agent-specific kernel language, which serves as the programming substrate for this loop. It should help the agent express its optimization ideas, implement the changes, and do better tuning.



NOTES

P9 · Design for agent failures · 6:15–7:00

To achieve this target, we have to understand the typical failure modes for tuning agents.

Firstly, the context gets used up very quickly. We keep adding code, profiler output and debugging messages. After several rounds, the agent may lose earlier attempts and repeat something it already tried.

Secondly, one optimization may need changes in many places. Take TMA as an example, the descriptor, swizzle, copy and synchronization have to work together. Missing one can break the kernel.

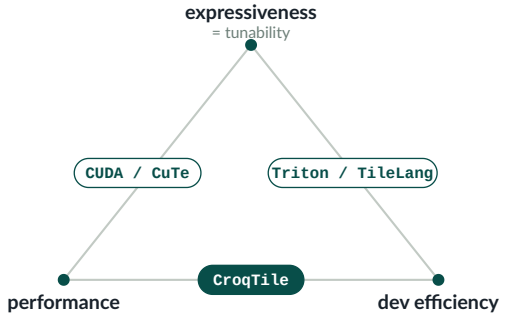
And lastly, the feedback can be hard to use. If the GPU just hangs, how does the agent know which statement to fix?

So these are the problems we want our language and compiler to address.

10 / 20

CroqTile: two co-design axes

EXPRESSIVENESS · PERFORMANCE · ITERATION COST



CUDA / CuTe	Explicit control	Long source · coupled edits
Triton / TileLang	Compact tile operations	Some structural controls hidden

- **Syntax–context co-design**
  Compact syntax and fewer coupled edit sites leave room for source, feedback and tuning history.
- **Compiler–harness co-design**
  Symbolic checks and structured diagnostics guide rejection, repair and GPU evaluation.

NOTES

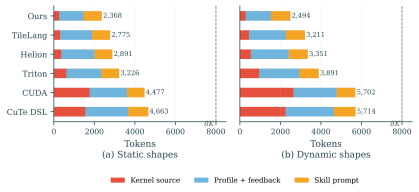
P10 · CroqTile: two co-design axes · 7:00–7:40

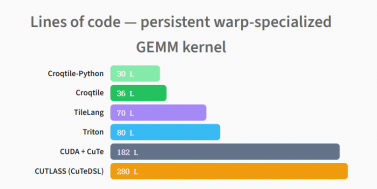
When we look into this problem, we found an impossible triangle when designing a kernel programming language: they are the expressiveness, performance and development efficiency.

CUDA and CuTe give us control and performance, but require a lot of code. Tile languages make programming easier, while hiding some structural choices that matter for tuning.

So our choice is to keep the performance controls explicit, and reduce the work around them. We do this along two axes: syntax–context co-design, to make the code compact for the agent; and compiler–harness co-design, to make failures easier to identify and repair.

Compact syntax preserves context







- **36 vs. 182 source lines**
  Same persistent, warp-specialized GEMM; CroqTile vs. CUDA + CuTe.
- **Room for tuning history**
  ≈2.37K vs. ≈4.48K tokens per static-shape attempt, including source, feedback and skills.

NOTES

P11 · Compact syntax preserves context · 7:40–8:20

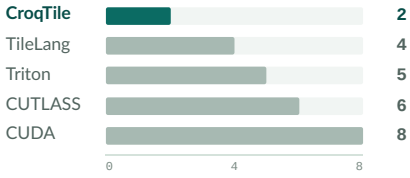
Let us look at the syntax–context part first. Here, we try to increase the information density and make the program source code much more compact without lose information.

This matters because the context has to hold much more than the source. We have skills and system prompts, profiler and compiler feedback, and we also need room for the agent's thinking.

And notably, if a bug emerges, many of these costs are multiplied. The agent reads the code again, gets more feedback, and spends more of its thinking budget on the repair.

parallel: one construct, three levels

Parallelism primitives



- **One binding, three levels**
  `block · group-4 · thread`
- **Explicit producer–consumer roles**
  `inthreads.async` separates TMA producers and MMA consumers.
- **Fewer coupled edit sites**
  Change the parallel binding without reconstructing thread-index calculations.

CroqTile | TileLang | CuTe / CUTLASS | CUDA

```
parallel {bm, bn}
  by [cdiv(M, WARP_M), cdiv(N, WARP_N)] : block {
  parallel p by 1 : group-4 {
    mma.row.row mc, ma, mb;
  }
  parallel {m, n}
    by [WARP_M, 2] : thread { ... }
}

// Producer-consumer warp specialization
parallel p1 by 2 : group-4 {
  inthreads.async (p1 == 0) { /* TMA producer */ }
  inthreads.async (p1 == 1) { /* MMA consumer */ }
}
```

NOTES

P12 · parallel: one construct, three levels · 8:20–8:55

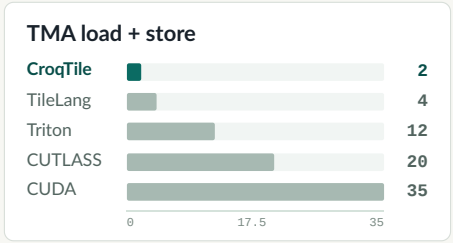
So how do we simplify the syntax without sacrificing performance? We first classify hardware controls and validate their performance impact.

We absorb controls with a clear preferred direction, and routine code that does not affect performance, into simpler syntax.

For controls with workload-dependent performance tradeoffs, we keep them explicit and provide more intuitive abstractions.

As you can see in the parallel example here, the work division stays explicit while the compiler handles the indexing and launch setup, which is just routing code.

tma: one statement per transfer



- **One statement per transfer**
  Source and destination specify the tile movement.
- **Swizzle as a modifier**
  `.swiz<128>` makes the layout choice explicit at the transfer.
- **Compiler-generated descriptor**
  The agent edits the transfer; the compiler keeps the derived descriptor consistent.

CroqTile | Triton | TileLang | CuTe / CUTLASS | CUDA

```
// LOAD · global → shared
tma.copy.swiz<128>
  lhs.subspan(WARP_M, TILE_K)
     .at(bm, iv_k) => lhs_s;

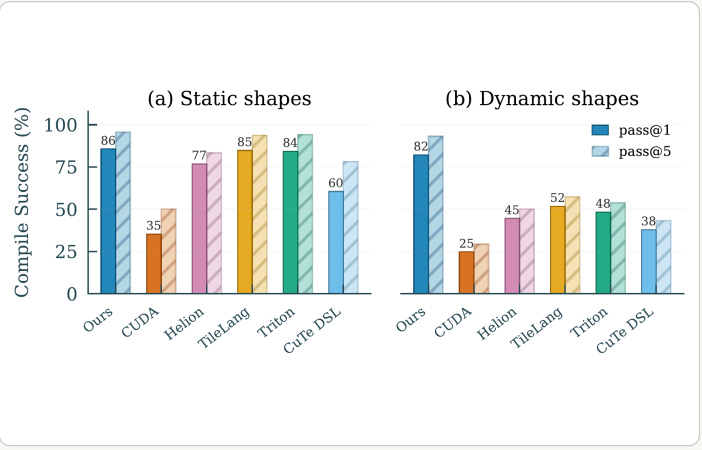
// STORE · shared → global
tma.copy out_s =>
  output.subspan(WARP_M, WARP_N)
        .at(bm, bn);
```

NOTES

P13 · tma: one statement per transfer · 8:55–9:20
The same idea applies to TMA data movement. Here, the syntax describes which tile moves where, and the compiler derives the corresponding descriptor.
But swizzling affects performance, so this choice stays explicit as a specifier.
As you can see in the load and store here, the agent can change the transfer or swizzle directly, without a seperate configuration statement, and the compiler keeps the descriptor consistent.

Compile success across shapes

PASS@1 / PASS@5 · STATIC + DYNAMIC



pass@1: 85.7% static · 82.1% dynamic
Dynamic shapes: Triton 48% in this evaluation.

1 Less coupled syntax

Each optimization touches fewer interdependent code sites.
Fewer details to keep consistent → fewer edit errors.

2 Symbolic inference

The type system and compiler infer shape relationships and check constraints.
Less manual shape bookkeeping, especially for dynamic shapes.

NOTES

P14 · Compile success across shapes · 9:20–9:55
So does this help the agent produce a program that compiles? Under the same agent, hardware and budget, CroqTile reaches 85.7 percent on the first attempt for static shapes.
Two things contribute here: fewer coupled edits in the syntax, and symbolic inference to work out shape relationships.
Of course, compiling successfully does not mean the result is correct. We still verify correctness, but fewer compile repairs leave more time for tuning.

Diagnostics that guide repair

CONSTRAINT · OBSERVED · LOCATION · REPAIR

```
<compile_result status="rejected" config="stg3,128x128" tokens="74">
  <constraint cat="shape-compat">
    DMA dim-0 mismatch. predicate: from.d0 == to.d0
  </constraint>
  <observed>
    from.d0 = (N / cdiv(N, 128))
    to.d0 = 128
    valid: to.d0 must equal from.d0
  </observed>
  <location file="spmm.co" line="58">
    tma.copy.async<full[stg]>.swiz<128>
      rhs.subspan(WN,TK).at(bn,k)
      => rhs_s.subspan(WN,TK).at(stg,0);
  </location>
  <repair priority="high" confidence="certain">
    Do: Append .zfill to the DMA — zero-fills tail tiles
      where N % 128 != 0.
    Why: Source tile < dest when N is not a multiple of 128.
    Alt: Set tile_n = cdiv(N,128)*128 so from.d0 == to.d0.
  </repair>
</compile_result>
```

1 Symbolic inference

Check shape compatibility and resource limits before launch.

2 Runtime assertions

Check constraints that depend on runtime values.

3 Compiler–harness interface

Pass **constraint, observed, location and repair** as structured fields.

4 Repair in the tuning loop

The harness returns the rejection to the agent; the diagnostic identifies the statement and corrective edit.

NOTES

P15 · Diagnostics that guide repair · 9:55–10:50

And when compilation does fail, what should we return to the agent? This brings us to another core idea of our work, the compiler and harness co-design.

We built a comprehensive symbolic inference engine and a type system atop of the primitive shape type. this gives us ability to do more compile-time checks and resource limits validation before launch, and we also use runtime assertions for constraints that depend on runtime values.

Let us follow the example on the left. The feedback tells us which constraint failed, what values we observed, where the statement is, and how to repair it. Here, the dimensions do not match, and the suggested repair is to add zero fill for the tail tile.

Prune before GPU execution

SPARSE GEMM · CONFIGURATION PRUNING



- **353 checks across 7 modules**
  Compiler checks cover shapes, resources and synchronization.
- **480 configurations → 102 viable**
  79% are pruned before GPU execution, mostly for register and shared-memory limits.
- **The harness measures viable candidates**
  Rejected candidates return with a repair hint; passing candidates proceed to correctness checks and timing.

NOTES

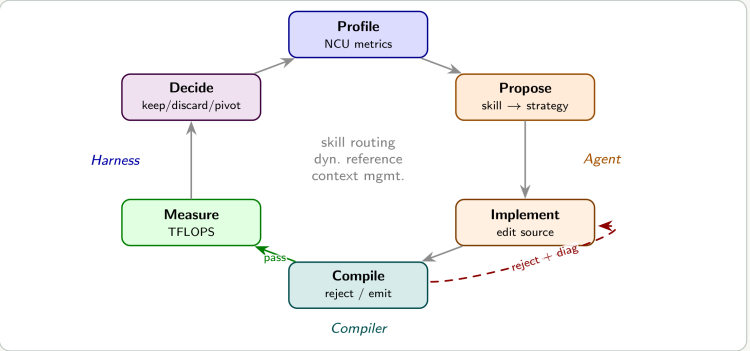
P16 · Prune before GPU execution · 10:50–11:25

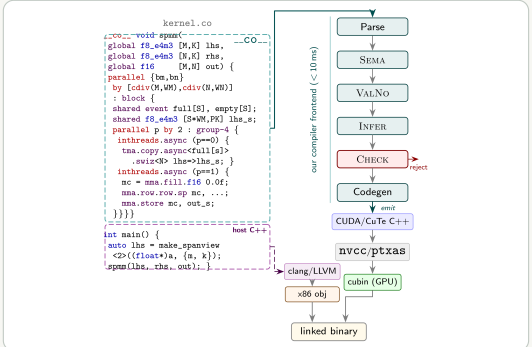
These checks also help us narrow the search before using the GPU. Candidates that exceed resource limits can be rejected early, with feedback telling the agent what to change.

The harness then checks correctness and measures performance for the remaining candidates.

So the compiler is part of the tuning loop: it helps the agent spend less time debugging invalid programs and more time finding faster kernels.

Close the autonomous loop





- **Syntax–context co-design**
  Compact source and local edits keep prior attempts and measurements available to the agent.
- **Compiler–harness co-design**
  Structured rejection feeds the next repair; valid candidates proceed to verification and measurement.

1.23× vendor libraries Geometric-mean speedup across evaluated workloads with vendor references.

NOTES

P17 · Close the autonomous loop · 11:25–12:00

Now let us put these pieces back into the tuning loop.

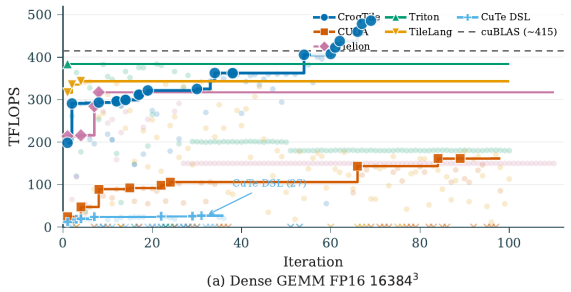
If compilation fails, the feedback goes back for repair. If it passes, we check correctness and measure performance. Skills and references provide the language rules and optimization ideas along the way. Across the evaluated workloads with vendor references, the resulting kernels achieve a geometric-mean speedup of 1.23 times. Let us look at two examples.

18 / 20

Tuning beyond cuBLAS

Dense FP16 GEMM

M = N = K = 16,384

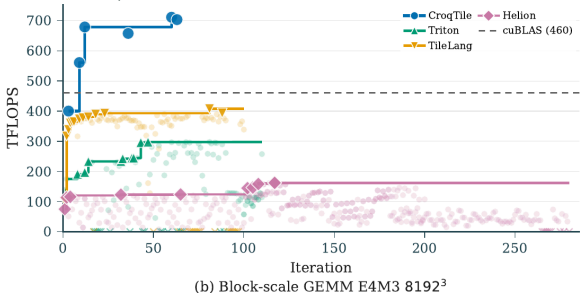


(a) Dense GEMM FP16 16384^3

Improves steadily, reaches cuBLAS, then exceeds it.

Block-scale E4M3 GEMM

M = N = K = 8,192



(b) Block-scale GEMM E4M3 8192^3

Crosses the vendor baseline early and continues improving.

- **Read the curves**
  Solid lines: running-best TFLOPS; dashed line: cuBLAS.
- **Shared evaluation protocol**
  H800 PCIe · Claude Sonnet 4.6 · same agent and per-shape time cap.

NOTES

P18 · Tuning beyond cuBLAS · 12:00–12:35

As you can see on the left, this is dense FP16 GEMM. The solid lines show the best throughput found so far, and the dashed line is cuBLAS. CroqTile gradually improves, reaches that line, and then goes beyond it.

On the right, for block-scale E4M3 GEMM, it crosses the vendor baseline earlier and keeps improving. Both use the same agent and per-shape time cap on H800.

Thank you



CroqTile is open source

Scan · CroqTile repo

ACCEPTED WORKS USING CROQTILE

- **SC'26** · Structured Sparse GEMM on Hopper via Analytical Modeling
- **SC'26** · Do We Need Tensor Cores for Stencil Computations? **Best Paper Nominee**
- **ATC'26** · An AI-Native GPU Kernel Programming Language for Autonomous Tuning
- **ATC'26** · SptcAttn: Long-Context Attention with Diagonal-Stripes Sparse Tensor Core
- **CGO'27** · Signal-Aware Liveness Analysis for Shared Memory in Warp-Specialized Kernels

NOTES

P19 · Thank you · 12:35–13:15

So the main idea is to make the language and compiler fit this iterative tuning process. Compact code helps the agent keep its history, explicit controls let it try optimizations, and reinforce the compiler checks and provide useful feedback helps the agent can repair mistakes accurately.

This language CroqTile is now open sourced, you can have to look if interested. And with this project, we have five accepted works here, that either describes the design of croqtile, or using croqtile for kernel programming. Please feel free to scan the code and have a try.

20 / 20 CroqTile in practice

CroqTile as a competition language

4th National Advanced Computing Technology Innovation Competition



Scan for competition details

ac-innovation.com

Kernel programming and optimization with CroqTile

Public competition information · China

NOTES

P20 · CroqTile in competition · 13:15–13:30

And also, CroqTile has been used as a programming language in China's fourth National Advanced Computing Technology Innovation Competition. You can scan the code for the public competition page. Thank you.